

AegisAI v1

Semantic Data Loss Prevention for AI Agents

AegisAI is an AI security gateway that inspects every response an agent produces, before it reaches the user, and detects when protected enterprise knowledge has been leaked semantically — paraphrased, summarised, or reconstructed — even when no sentence is copied and no regex pattern fires.

This document describes **v1**: the scope that is realistically buildable, deployable, and demonstrable within a 24-hour build window. Features that require more time are listed explicitly under Roadmap (Section 9), not claimed as built.

Prepared for: Agentic AI Task — AI Governance (Aivar Innovations)

Prepared by: Sudhan | B.Tech AI & Data Science

1. Problem

Traditional DLP tools catch exfiltration only when it matches a known pattern — a card number, an email, a keyword. An LLM can leak the same information without producing that pattern at all.

Protected document: "John is leading Project Orion with an annual budget of \$18 million."

LLM output: "The Orion initiative is managed by John and receives approximately eighteen million dollars every year."

No sentence copied, no keyword match, no regex trigger — but the confidential fact has still left the system. Pattern-based DLP compares words; this problem requires comparing meaning.

2. What AegisAI v1 Does

A two-stage pipeline scores every agent output against a protected knowledge vault, then a lightweight policy engine turns that score into a governance action. Every decision — allowed or not — is written to an immutable, queryable audit log.

Stage 1 — Similarity Scorer (recall-oriented, cheap, runs on everything)

- Vault documents are chunked and embedded once at ingest time; agent outputs are embedded per request.
- Cosine similarity against the nearest vault chunks produces a fast score — deliberately over-sensitive, so it should not miss a real paraphrase.

Stage 2 — Factual Overlap Detector (precision-oriented, only runs above threshold)

- Outputs that clear the stage-1 threshold, plus their top matching vault chunk(s), go to an LLM judge.
- The judge returns a structured verdict: does the output assert specific facts only present in the vault, however reworded? It defeats deliberate obfuscation because it checks facts, not phrasing.

Governance Policy Engine (new in v1 — cheap to add, high narrative value)

- The combined risk score (0-100) is routed through a simple threshold policy: **<30 Allow, 30-60 Redact, 60-85 Human Review, >85 Block.**
- This is a router on top of a number you already compute — no separate scoring model, no extra latency of note.

Explainability (new in v1 — free, comes from the judge's own output)

- The judge is prompted to return matched facts, the matched document, and a one-line reason alongside its verdict.
- This is surfaced as-is in the API response and audit record — not a separate 'explainability engine,' just a well-shaped judge prompt.

3. System Architecture

AegisAI v1 — Architecture (24h Scope)

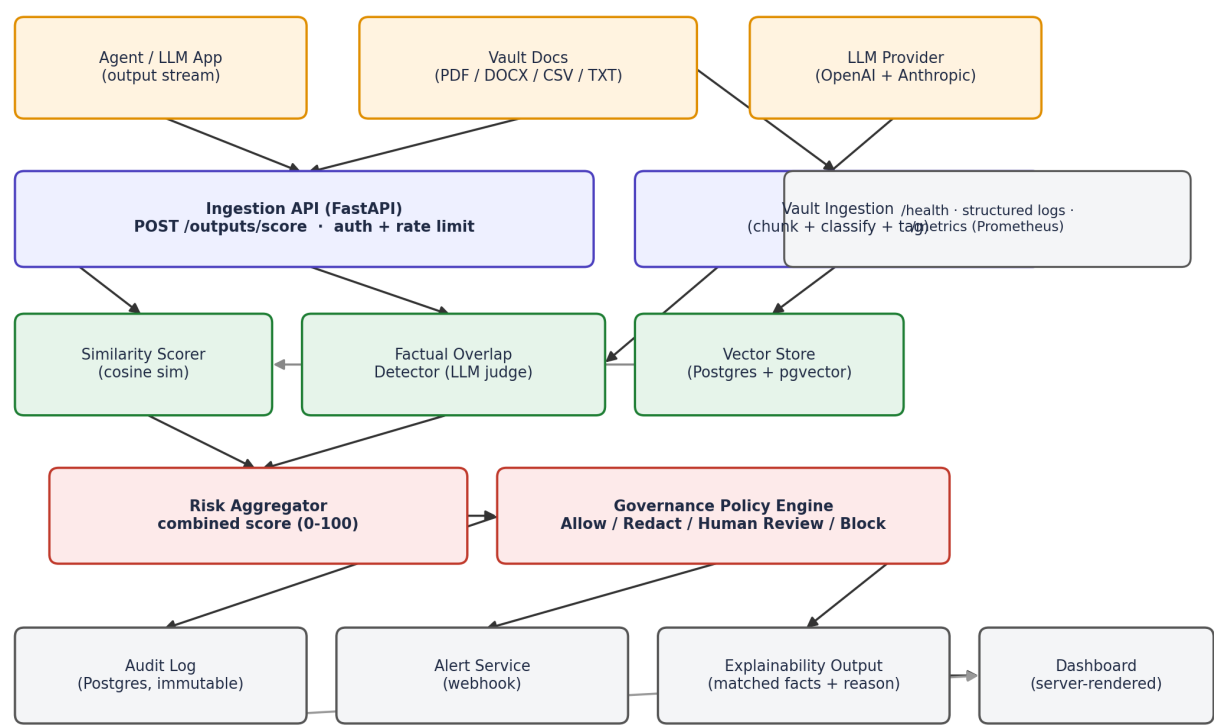


Figure 1 — v1 request flow: score → aggregate → govern → log/alert/explain.

Component Responsibilities

Component	Responsibility
Ingestion API	Receives agent outputs, authenticates the caller (API key per agent), rate-limits, hands off to the pipeline.
Vault Ingestion	Accepts PDF / DOCX / CSV / TXT, extracts text (pypdf, python-docx, csv), chunks it, tags it with a sensitivity score.
Vector Store	Postgres + pgvector — one database for vault embeddings, audit log, and app state; no second stateful store.
Similarity Scorer	Cosine similarity between output embedding and nearest vault chunks.
Factual Overlap Detector	LLM-judge call (Claude); structured JSON verdict + matched facts + confidence.
Risk Aggregator	Combines both signals into one score; owns the calibration thresholds.
Governance Policy Engine	Maps the risk score to Allow / Redact / Human Review / Block.
Audit Log	Append-only Postgres table — every request, both scores, verdict, policy action, regardless of outcome.
Alert Service	Fires a webhook when policy action is Block or Human Review.
Dashboard	Server-rendered read view (Jinja2) over the audit log — recent flags, score distribution, eval report.

4. Tech Stack (v1)

Layer	Choice	Why it fits 24h
-------	--------	-----------------

API framework	FastAPI (Python)	Async-native, handles concurrent scoring, auto-generates OpenAPI docs
LLM provider	Anthropic API (judge) + OpenAI embeddings	Real, live calls — satisfies the 'not mocked' criterion; a thin provider interface
Vector store	Postgres + pgvector	One DB instance for everything; avoids a second service to configure and maintain
File parsing	pypdf, python-docx, csv (stdlib)	All are one-line extraction calls — genuinely cheap, keep this from the original
Auth	API key per calling agent	Enough to attribute audit records to a caller; full RBAC is out of scope for now
Containerization	Docker + docker-compose (api + postgres)	One-command local run; same image ships to the cloud target.
Deployment	Railway or Render (Docker deploy)	Public HTTPS URL, managed Postgres, TLS, minimal ops overhead.
Observability	structlog JSON logs + /health + /metrics (Prometheus)	Directly satisfies logging, error handling, health check.'
Dashboard UI	Server-rendered Jinja2 page	Same data a React dashboard would show, ~45 minutes instead of half a day
Alerts	One generic webhook (Slack-compatible)	One integration, not three — Teams/Email dropped from v1.

5. Data Model

```
vault_documents
  id, title, source_type (pdf/docx/csv/txt), sensitivity_tag, owner, created_at

vault_chunks
  id, document_id (FK), text, embedding (vector), chunk_index

scored_outputs  (audit log – append only)
  id, agent_id, session_id, output_text, similarity_score,
  matched_chunk_ids[], judge_verdict, judge_confidence, matched_facts (jsonb),
  risk_score, policy_action, created_at

alerts
  id, scored_output_id (FK), policy_action, delivered_at, status
```

6. API Design

Endpoint	Method	Purpose
/v1/vault/documents	POST	Upload + chunk + embed a PDF/DOCX/CSV/TXT protected document.
/v1/outputs/score	POST	Score a single agent output; returns similarity score, judge verdict, risk score, policy action
/v1/outputs/{id}	GET	Fetch the full scoring record for a past output.
/v1/alerts	GET	List recent Block / Human-Review events.
/v1/eval/run	POST	Run the 20-case labelled test suite (10 normal / 5 paraphrased / 5 borderline); returns pre
/health	GET	Liveness/readiness — DB connection, provider reachability.
/metrics	GET	Prometheus counters: requests scored, actions by type, judge latency, provider errors.

7. Evaluation Plan

Success Criterion	How It's Verified
-------------------	-------------------

Similarity scorer ranks paraphrases above unrelated outputs	1. Eval/run compares score distributions for 10 normal vs 5 paraphrased cases.
Factual overlap: 4/5 paraphrased cases correctly flagged	2. Judge verdicts checked against ground-truth labels in the eval report.
False positive rate < 20% on normal outputs	Eval report computes FP rate at the calibrated threshold over the 10 normal cases.
Detection survives deliberate obfuscation	5 paraphrased cases are written to reconstruct facts without quoting; judge evaluates

8. Production Readiness Checklist

- **Deployed, not localhost:** Docker image on Railway/Render, public HTTPS URL.
- **Concurrency & persisted state:** async FastAPI + Postgres connection pool; nothing held only in memory.
- **Real LLM provider:** live Anthropic + OpenAI calls, not mocked responses.
- **Logging & error handling:** structured JSON logs per request; judge failures degrade to a similarity-only tier with a flag, not a crash.
- **Health check:** /health checks DB + provider reachability, used as the platform readiness probe.
- **Secrets:** API keys via environment variables / platform secret store.

9. Roadmap — Explicitly Not in v1

These were part of the original AegisAI concept but are cut from the 24h build. Listing them here, clearly labelled as future work, is deliberate: it signals scope discipline to a reviewer instead of inviting a follow-up question you can't answer live.

Feature	Why it's out of scope for v1
React + Tailwind + Chart.js dashboard	4-6 hrs of frontend work for the same data a server-rendered page shows in ~45 minutes.
Role-Based Access Control (RBAC)	Needs a user/role/permission model; a single API key per agent is enough to demo governance.
Session replay	Meaningful UX on top of session history — needs real usage data and dedicated screen time.
Department-wise leak trends, historical violation trends	Needs weeks of real traffic to be meaningful; a 24h-old system has no history to trend.
Multi-tenant SaaS support	Tenancy, isolation, and billing hooks are days of work, not hours.
Adaptive risk threshold learning	Needs a labelled feedback loop and retraining logic beyond a single build cycle.
Prompt injection detection	A distinct problem statement (PS-3.2) in the same brief — kept separate rather than conflated.
Slack + Teams + Email alert integrations	v1 ships one generic webhook; each additional channel is its own integration.
Full compliance mapping engine (ISO 42001, NIST, GDPR, etc.)	Needs a RBAC + framework tag per policy rule; a real mapping engine is a separate build.
Sentry / full observability stack	Structured logs + /health + /metrics already satisfy the brief's observability bar.

10. 24-Hour Build Timeline

Time Block	Focus
Hour 0-2	Repo scaffold, Docker + docker-compose (FastAPI + Postgres/pgvector), /health endpoint, deploy skeleton early so
Hour 2-5	Vault ingestion: PDF/DOCX/CSV/TXT parsing, chunking, embedding write path; build the synthetic reference vault.

Hour 5-9	Stage 1 similarity scorer + vector search; /v1/outputs/score returns a raw similarity score.
Hour 9-13	Stage 2 LLM judge integration (prompt design, structured JSON output, retry/error handling).
Hour 13-15	Risk aggregator + Governance Policy Engine (threshold routing to Allow/Redact/Review/Block).
Hour 15-17	Calibration against the 20-case labelled test suite until success criteria pass.
Hour 17-19	Audit log, one webhook alert, /v1/eval/run report endpoint, /metrics.
Hour 19-21	Server-rendered dashboard (recent flags, score distribution, eval report).
Hour 21-23	Deploy final build, redeploy after calibration changes, smoke-test every endpoint against the live URL.
Hour 23-24	README, this proposal, and a short recorded demo as backup evidence of a live, deployed system.

AegisAI v1 — scoped solution proposal for PS-5.3, Unit 5 (Agentic AI Security), Agentic AI Task (Aivar Innovations, June 2026).